

Frontend & Backend Professional Project Structure Guide

Beere Kesava & Brothers Silks ERP

Purpose: Turn the current Figma-Make React prototype (all logic in React Context, one giant file per feature) into a clean, layered, professional codebase — and pair it with a real NestJS backend using the same folder-structure discipline.

Companion document: See 'Backend Architecture Design' PDF for the full data model and API contract. This document focuses on *folder structure and code organization* for both sides.

Part A — Frontend Restructure

A.1 What's wrong today

Aspect	Current (prototype)	Target (professional)
File size	PaymentsPage.tsx ~4300 lines, WeaversPage.tsx ~2400 lines, split into Context.tsx, components, layouts, providers, routes, types, utils, etc.	Each file < 200 lines, split into Context.tsx, components, layouts, providers, routes, types, utils, etc.
Data layer	React Context holds hardcoded seed arrays	Context state fully holds server data fetched via React Query; no bus
API access	Components call Context mutators directly (addBatch, createBatch, etc.)	Added Order, applyWeaver, etc. fetch calls; components use React Query
Types	Interfaces defined inline inside each *Context.tsx	Shared types/ package, mirroring backend DTOs, imported everywhere
Auth	localStorage flag, no real verification	Real JWT stored in memory/httpOnly cookie, refreshed via api/auth, g
Routing/pages	Pages already split per role — good, keep this	Keep the *Layout.tsx per-role pattern; pages become thin containers

A.2 Target folder structure

```
frontend/
├── src/
│   ├── main.tsx
│   ├── app/
│   │   ├── App.tsx
│   │   ├── layouts/
│   │   ├── providers/
│   │   └── routes/
│   │       └── # routes only
│   │           └── # AdminLayout, WeaverLayout, ... (keep as-is)
│   │               └── # QueryClientProvider, AuthProvider, ThemeProvider
│   │                   └── # route config per role (extracted from App.tsx)
│   ├── features/
│   │   └── # ONE folder per business domain (replaces *Context.tsx sprawl)
│   │       ├── auth/
│   │       │   ├── api.ts
│   │       │   ├── hooks.ts
│   │       │   ├── store.ts
│   │       │   ├── types.ts
│   │       │   └── # requestOtp(), verifyOtp(), refresh()
│   │       │       └── # useAuth(), useLogin()
│   │       │           └── # zustand/jotai auth store (role, token, accessLevel)
│   │       ├── batches/
│   │       │   ├── api.ts
│   │       │   ├── hooks.ts
│   │       │   └── # useBatches(), useCreateBatch(), useAssignRow()
│   │       ├── components/
│   │       │   ├── BatchCreationPage.tsx
│   │       │   ├── BatchRowTable.tsx
│   │       │   ├── BatchStatusBadge.tsx
│   │       │   └── types.ts
│   │       ├── weavers/
│   │       │   ├── api.ts
│   │       │   ├── hooks.ts
│   │       │   ├── components/
│   │       │   │   ├── WeaversPage.tsx
│   │       │   │   ├── WeaverAnalyticsCard.tsx
│   │       │   │   ├── WeaverProductionTable.tsx
│   │       │   │   └── WeaverImportModal.tsx
│   │       │   └── # thin: layout + composition only
│   │       │       └── # Excel import UI only
│   │       ├── utils/
│   │       │   ├── excelImport.ts
│   │       │   ├── types.ts
│   │       │   └── # pure parsing/matching logic, unit-testable
│   │       ├── payments/
│   │       │   ├── api.ts
│   │       │   ├── hooks.ts
│   │       │   ├── components/
│   │       │   │   ├── PaymentsPage.tsx
│   │       │   │   ├── WeaverPaymentsTab.tsx
│   │       │   │   ├── VendorPaymentsTab.tsx
│   │       │   │   ├── InvoicesTab.tsx
│   │       │   │   └── ExcelUploadModal.tsx
│   │       │   └── # same split as weavers: page shrinks from 4300 → ~200 lines
│   │       │       ├── utils/
│   │       │       │   ├── weaverMatching.ts
│   │       │       │   └── vendorMatching.ts
│   │       │       └── types.ts
│   │       ├── quality-control/
│   │       ├── finishing/
│   │       ├── inventory/
│   │       ├── bulk-orders/
│   │       ├── design-library/
│   │       ├── factory-ooms/
│   │       └── suppliers-vendors/
```

```

■ ■ ■■■ purchase-orders/
■ ■ ■■■ firms/
■ ■ ■■■ customers/
■ ■ ■■■ users-admin/
■ ■ ■■■ audit-log/
■ ■ ■■■ notifications/
■ ■
■ ■■■ shared/
■ ■ ■■■ components/
■ ■ ■■■ ui/
■ ■ ■■■ hooks/
■ ■ ■■■ utils/
■ ■
■ ■■■ api/
■ ■ ■■■ client.ts
■ ■ ■■■ queryKeys.ts
■ ■
■ ■■■ types/
■ ■■■ constants/

# DateFilterBar, Pagination, SectionNavigator (already shared, keep)
# shadcn/MUI primitives
# useDebounce, usePagination, etc.
# id formatting, currency, date helpers

# axios/fetch instance, interceptors (auth header, 401 refresh)
# centralized React Query key factory

# cross-feature shared types (mirrors backend DTOs)
# imageData.ts, weaverImages.ts (already exists, keep)

```

A.3 Migration approach (incremental, not big-bang)

- Pick one feature at a time, starting with the two worst offenders: **payments** and **weavers**.
- Extract pure logic first (Excel parsing/matching) into `utils/*.ts` with unit tests — this alone shrinks the mega-files a lot with zero behavior change.
- Introduce `features/<name>/api.ts` + React Query hooks that, for now, just wrap the existing Context calls — this creates the seam you'll later point at the real backend without touching components again.
- Once backend endpoints exist (see companion PDF), swap the `api.ts` implementation from Context-backed to fetch-backed. Components/hooks don't change.
- Delete each `*Context.tsx` only after its feature folder fully replaces it — keep the app running at every step.

Part B — Backend Structure (NestJS)

B.1 Why NestJS over Express here

- ~15+ domain modules with heavy RBAC (6 roles × access levels) — Nest's Guards give you this for free; Express means hand-rolling middleware per route.
- Deep cross-module relations (batch → saree → QC → finishing → dispatch → invoice → payment) — Nest's DI makes services composable and testable in isolation.
- Cross-cutting rules (audit logging, money-field filtering, ID generation) map directly to Nest Interceptors/Pipes instead of ad-hoc Express middleware chains.
- TypeScript-first and pairs naturally with Prisma, matching your existing TS frontend types.

B.2 Target folder structure

```
backend/
  ■■■ src/
  ■   ■■■ main.ts                # bootstrap, global pipes/filters
  ■   ■■■ app.module.ts         # root module, imports every feature module
  ■   ■
  ■   ■■■ config/
  ■   ■   ■■■ env.validation.ts  # joi/zod schema for required env vars
  ■   ■   ■■■ configuration.ts
  ■   ■
  ■   ■■■ common/
  ■   ■   ■■■ decorators/
  ■   ■   ■   ■■■ roles.decorator.ts    # @Roles('admin', 'superadmin')
  ■   ■   ■   ■■■ current-user.decorator.ts
  ■   ■   ■■■ guards/
  ■   ■   ■   ■■■ jwt-auth.guard.ts
  ■   ■   ■   ■■■ roles.guard.ts
  ■   ■   ■   ■■■ access-level.guard.ts
  ■   ■   ■■■ interceptors/
  ■   ■   ■   ■■■ audit-log.interceptor.ts
  ■   ■   ■   ■■■ money-field-filter.interceptor.ts
  ■   ■   ■■■ filters/
  ■   ■   ■   ■■■ http-exception.filter.ts
  ■   ■   ■■■ pipes/
  ■   ■   ■   ■■■ validation.pipe.ts
  ■   ■   ■■■ utils/
  ■   ■       ■■■ id-generator.ts      # PO-2026-001, MIR-2026-001, EMP-001
  ■   ■       ■■■ jari-conversion.ts   # grams <-> reels <-> buns
  ■   ■
  ■   ■■■ prisma/
  ■   ■   ■■■ schema.prisma
  ■   ■   ■■■ migrations/
  ■   ■   ■■■ prisma.service.ts
  ■   ■
  ■   ■■■ modules/
  ■   ■   ■■■ auth/
  ■   ■   ■   ■■■ auth.module.ts
  ■   ■   ■   ■■■ auth.controller.ts
  ■   ■   ■   ■■■ auth.service.ts
  ■   ■   ■   ■■■ otp.service.ts      # SMS provider integration
  ■   ■   ■   ■■■ dto/
  ■   ■   ■       ■■■ request-otp.dto.ts
  ■   ■   ■       ■■■ verify-otp.dto.ts
  ■   ■   ■■■ users/
  ■   ■   ■■■ vendors/
  ■   ■   ■■■ suppliers/
  ■   ■   ■■■ purchase-orders/
  ■   ■   ■   ■■■ purchase-orders.module.ts
  ■   ■   ■   ■■■ purchase-orders.controller.ts
  ■   ■   ■   ■■■ purchase-orders.service.ts
  ■   ■   ■   ■■■ dto/
  ■   ■   ■■■ purchase-requests/
  ■   ■   ■■■ design-library/
  ■   ■   ■■■ batches/
  ■   ■   ■■■ factory-looms/
  ■   ■   ■■■ material-issues/
  ■   ■   ■■■ qc/
  ■   ■   ■■■ finishing/
```

```

■ ■ ■ quotations/
■ ■ ■ bulk-orders/
■ ■ ■ customers/
■ ■ ■ inventory/
■ ■ ■ sarees/
■ ■ ■ dispatch/
■ ■ ■ weaver-payments/
■ ■ ■ vendor-payments/
■ ■ ■ invoices/
■ ■ ■ firms/
■ ■ ■ rates-pricing/
■ ■ ■ notifications/
■ ■ ■ audit-log/
■ ■ ■ reports/
■ ■ ■ uploads/ # S3 pre-signed URL issuing
■ ■
■ ■ ■ jobs/ # BullMQ processors
■ ■ ■ excel-match.processor.ts # weaver/vendor payment matching
■ ■ ■ overdue-scan.processor.ts # Day-45 bulk-order payment alert
■ ■ ■ notification-fanout.processor.ts
■ ■
■ ■ ■ test/ # e2e tests, separate from unit tests colocated per module
■
■ ■ ■ docker-compose.yml # postgres, redis, minio (local S3)
■ ■ ■ Dockerfile
■ ■ ■ .env.example
■ ■ ■ nest-cli.json
■ ■ ■ package.json

```

B.3 Per-module file convention

Every module in `modules/` follows the same 4-file shape, so any engineer can predict where to find things:

- `*.module.ts` — wires controller + service + Prisma into Nest's DI container
- `*.controller.ts` — routes only: decorators for auth/roles, delegates to service, no business logic
- `*.service.ts` — business logic, DB calls via `PrismaService`, one service per bounded context
- `dto/` — request/response validation classes (class-validator), one file per endpoint payload

B.4 Getting started — commands

```

# 1. Scaffold
npm i -g @nestjs/cli
nest new backend
cd backend

# 2. Core deps
npm i @prisma/client prisma @nestjs/jwt @nestjs/passport passport passport-jwt
npm i @nestjs/bullmq bullmq ioredis
npm i class-validator class-transformer
npm i @aws-sdk/client-s3 @aws-sdk/s3-request-presigner

# 3. Prisma
npx prisma init
# paste schema from the Backend Architecture Design PDF (section 4) into prisma/schema.prisma
npx prisma migrate dev --name init

# 4. Local infra
docker compose up -d # postgres + redis + minio

# 5. Run
npm run start:dev

```

Both structures are designed to mirror each other 1:1 (features/<name> on the frontend ↔ modules/<name> on the backend) so any endpoint or feature is easy to trace across the stack.